

Database Management Systems - Unit 4

8 topics · 653 words · 4 Oct 2026

Contents

Transactions	2
ACID Properties	2
Transaction States	2
Concurrency Control	3
Lock-Based Protocols	3
Deadlock Handling	3
Recovery	4
Log-Based Recovery	4
Checkpoints	4

Transactions

Key terms **logical** · **unit** · **takes** · **complete** · **entirely** · **effect** · **transaction** · **work**

A **transaction** is a **logical unit** of **work** that **takes** the database from one consistent state to another. Every transaction must either **complete entirely** or have no **effect** at all.

ACID Properties

Key terms **isolation** · **transaction** · **database** · **properties** · **even** · **fails** · **atomicity** · **consistency**

Every **transaction** in a relational **database** must satisfy four **properties**, known together as the **ACID** properties. They guarantee that the database stays correct **even** when many users work at once or the system **fails** halfway through an update.

- **Atomicity**: either all operations of the transaction are reflected in the database or none are.
- **Consistency**: execution of a transaction in **isolation** preserves the consistency of the database.
- **Isolation**: each transaction is unaware of other transactions executing concurrently.
 - Enforced by the concurrency control component.
 - Weaker isolation levels trade safety for speed.
- **Durability**: after a transaction completes, its changes persist even if the system fails.

The recovery manager is responsible for atomicity and durability, while the concurrency control manager is responsible for isolation.

Transaction States

Key terms **failed** · **committed** · **partially** · **after** · **aborted** · **state** · **transaction** · **active**

A **transaction** moves through a fixed set of **states** during its lifetime. The log records every state change so that the system can recover **after** a crash.

1. **Active**: the initial state, while the transaction is executing its statements.
2. **Partially committed**: after the final statement has been executed.
3. **Failed**: after the discovery that normal execution can no longer proceed.
4. **Aborted**: after the transaction has been rolled back and the database restored.
5. **Committed**: after successful completion.

State	Can move to
Active	Partially committed, Failed
Partially committed	Committed, Failed
Failed	Aborted

Concurrency Control

Key terms **same** · **interfere** · **result** · **some** · **serial** · **order** · **concurrency** · **ensures**

Concurrency control **ensures** that transactions running at the **same** time do not **interfere** with each other and that the final **result** is the same as **some serial order**.

Lock-Based Protocols

Key terms **lock** · **two-phase** · **locking** · **locks** · **item** · **read** · **write** · **transaction**

A **lock** is a mechanism to control concurrent access to a data **item**. A **transaction** must hold a lock on an item before it can **read** or **write** that item, and the lock manager decides whether a request can be granted.

Lock modes

- **Shared lock (S)**: the transaction can read the item but cannot write it.
- **Exclusive lock (X)**: the transaction can both read and write the item.

Two-phase locking

The **two-phase locking** protocol ensures conflict serializable schedules. In the growing phase a transaction may obtain locks but may not release any. In the shrinking phase it may release locks but may not obtain any new ones.

- Strict two-phase locking holds all exclusive locks until the transaction commits.
- Rigorous two-phase locking holds all locks until commit.

Deadlock Handling

Key terms **younger** · **transaction** · **set** · **one** · **deadlock** · **older** · **cycle** · **back**

A system is in a **deadlock** state if there is a **set** of **transactions** such that every transaction in the set is waiting for another transaction in the set. Deadlocks can be prevented in advance or detected and broken after they occur.

Prevention schemes

- **Wait-die**: an **older** transaction waits for a **younger one**, a younger one is rolled **back**.
- **Wound-wait**: an older transaction wounds a younger one and forces it to roll back.

Detection and recovery

The system maintains a wait-for graph and periodically searches it for a **cycle**. When a cycle is found, a victim transaction is chosen and rolled back to break the deadlock.

Recovery

Log-Based Recovery

Key terms **log** · **update** · **stable** · **storage** · **database** · **restores** · **failure** · **using**

The recovery system **restores** the **database** to a consistent state after a **failure**, **using** information stored in a **log** on **stable storage**.

The **log** is a sequence of log records that describe every **update** made to the database. With **write-ahead logging**, the log record for an update must reach stable storage before the changed data item is written to disk.

1. **Analysis**: find the transactions that were active at the time of the crash.
2. **Redo**: repeat history by reapplying every logged update.
3. **Undo**: roll back the transactions that never committed.

Checkpoints

Key terms **checkpoint** · **records** · **log** · **recovery** · **point** · **buffered** · **modified** · **pages**

A **checkpoint records** a **point** at which all **buffered log** records and **modified pages** have been written to disk. During **recovery** the system only needs to examine the log from the most recent checkpoint onwards, which greatly reduces recovery time.

- Fuzzy checkpoints allow transactions to keep running while the checkpoint is taken.
- The checkpoint record lists the transactions active at that moment.